TITLE

Project Proposal

Matthew La
Bachelor of Science in Computer Science
The University of Bath

November 2, 2026

Contents

1 Problem Description **1**
1.1 Aims . . . 1
1.2 Objectives . . . 2

2 Requirements Specification **2**
2.1 Functional Requirements . . . 2
2.1.1 Language . . . 2
2.1.2 Compiler and Runtime Enforcement . . . 3
2.1.3 Static Analysis . . . 3
2.1.4 Evaluation . . . 3
2.2 Non-Functional Requirements . . . 3
2.3 Out of Scope . . . 4

3 Project Plan **4**
3.1 Gantt Chart . . . 4
3.2 Tasks and Milestones . . . 4
3.3 Risk Assessment . . . 4

4 Resources **4**

1 Problem Description

Since the public availability of GitHub Copilot and ChatGPT in 2022, AI-assisted code generation has become a staple of software engineering, signalling a shift away from hand-written code. In 2026, Stack Overflow's Developer Survey reported that 83% of respondents used AI tools at work (Stack Overflow, 2026). Furthermore, GitHub reports that the proportion of pull requests that are initially drafted by AI has more than doubled between March and September 2026 (GitHub, 2026).

However, studies have shown that much AI-generated code can be incorrect, insecure, or both. For example, Vero et al. (2025) show that even OpenAI o1 may perform poorly when writing realistic software, with only 62% code correctness. Additionally, roughly half of these correct programs still contained vulnerabilities.

Trust in LLM output therefore remains a significant factor for AI adoption. Indeed, while 87% of Stack Overflow survey respondents report some level of trust in AI's outputs, 48% trust AI only when they can easily verify its output (Stack Overflow, 2026).

One of the most common methods for developers to carry out this kind of verification for their software is unit testing. Even so, test-passing AI-generated software patches still contain errors at a noticeable rate, with 29.6% of these patches behaving differently from the developers' real fix, and 28.6% of these divergences containing incorrect code (Wang, Y., Pradel, and Liu, 2025).

More fundamentally, tests are a finite set of hand-picked inputs that inevitably miss unconsidered edge cases. As Dijkstra (1970, p. 7) observed, "program testing can be used to show the presence of bugs, but never to show their absence". Dijkstra's own later solution to this problem was to explicitly prove a program's correctness (Dijkstra, 1972).

Hoare (1969) gave this idea a formal basis, and formal verification can now guarantee that a program meets its specification for every possible input. However, these guarantees are expensive. The seL4 microkernel's roughly 8,700 lines of C required around 200,000 lines of Isabelle/HOL proof (Klein et al., 2009), and a survey of 216 practitioners identifies scalability and the expertise required as key barriers to adoption (Gleirscher and Marmsoler, 2020).

Yet the idea at the core of verification, stating explicitly what a program should do and checking the code against that statement mechanically, need not carry this cost. Design by contract (Meyer, 1992) attaches such specifications to each function as preconditions and postconditions, making intended behaviour part of the code rather than leaving it implicit in a set of tests. This suits AI-generated code particularly well: the human states intent as contracts, the LLM writes the implementation, and a checker holds one to the other, the shift from writing code to specifying it that Kleppmann (2025) argues AI will make necessary.

1.1 Aims

Runtime contract systems, such as that of Findler and Felleisen (2002), only detect violations on inputs that are actually executed, sharing the blind spot of tests. Static verifiers such as Dafny (Leino, 2010) cover every input, but require the programmer to supply specifications and annotations until the verifier is satisfied. Gradual verification (Bader, Aldrich, and Tanter, 2018) addresses the weaknesses of each by proving what it can statically and checking the remainder at runtime. For LLM-generated code, existing work uses contracts mainly to guide generation through the prompt (Wang, S., 2026), and while runtime checking that feeds violations back to the LLM has been proposed as a future direction, whether enforcing contracts helps beyond simply stating them remains untested.

This project addresses this question. Its aims are to:

- Design and implement a subset of JavaScript extended with contracts, which compiles back to the full language, checking each contract at compile time where possible and at runtime otherwise.

- Determine how many contracts can be decided at compile time, and at what runtime cost the remainder are enforced.
- Evaluate whether enforced contracts help an LLM coding agent produce correct code, compared with the same constraints given as comments or not at all, so that the effect of enforcement is separated from the information the constraints carry.

JavaScript is chosen for two reasons. It is the most used programming language in the Stack Overflow Developer Survey, used by 62% of respondents in 2026 (Stack Overflow, 2026), and LLMs are also highly familiar with it, so unfamiliar syntax should not confound the evaluation.

The evaluation is scoped to the function level, the granularity at which contracts are written (Meyer, 1992). Extending it to larger, multi-function programs is left as future work.

1.2 Objectives

The aims will be met through the following objectives:

1. Define the syntax and semantics of a contract language for a subset of JavaScript expressive enough to implement the benchmark tasks.
2. Formalise the static analysis for the language, defining the abstract domain, how each construct transforms it, and how loops are handled, with an argument that its results are sound with respect to the language's semantics.
3. Implement a compiler that transpiles the language to JavaScript with runtime contract checks, validated by a test suite.
4. Implement a static checker that proves or refutes contracts at compile time, validated by testing that no proven contract fails at runtime.
5. Measure which contracts can be decided at compile time on independently selected tasks, and characterise those that cannot.
6. Compare an LLM agent's hidden-test pass rate with enforced contracts, the same constraints as comments, and no constraints.

2 Requirements Specification

Each requirement is given one of three priorities: **must** (essential), **should** (preferable but not essential), or **may** (a desirable extension).

2.1 Functional Requirements

2.1.1 Language

- The language **must** support functions, `let`/`const`, conditionals, loops, numbers, booleans, strings and arrays.
- The language **must** support preconditions (`requires`), postconditions (`ensures`) and references to the return value (`result`).
- The language **should** support `old()` expressions and loop invariants.
- The language **may** support quantified contracts over arrays, checked at runtime only.

2.1.2 Compiler and Runtime Enforcement

- The compiler **must** transpile programs to readable JavaScript.
- The compiler **must** insert runtime checks for contracts, reporting violations with blame (caller or function).
- The compiler **must** provide a command-line interface for compiling and running programs.
- The compiler **should** support three modes: no contracts, all runtime checks, and gradual checking.

2.1.3 Static Analysis

- The checker **must** classify contracts in loop-free code as proven, refuted or unknown using interval analysis (Møller and Schwartzbach, 2026), which tracks a range of possible values for each numeric variable, starting from the function's preconditions and merging ranges where branches join.
- The checker **should** extend this to loops using widening, narrow ranges using branch conditions, and track array lengths.
- The checker **may** decide relational contracts in loop-free functions by translating each function and its contracts into a logical formula and checking it with the Z3 SMT solver (Moura and Bjørner, 2008), reporting counterexamples for refuted contracts.

2.1.4 Evaluation

- The evaluation **must** use tasks from an existing function-level benchmark, with the benchmark's own tests hidden from the LLM and used to judge correctness.
- The evaluation **must** fix all contracts before any experimental runs, writing them from each task's description and validating them against its reference solution.
- The evaluation **must** measure the proportion of incorrect LLM-generated solutions that contracts detect, how many at compile time, and how many correct solutions are wrongly flagged.
- The evaluation **must** measure runtime overhead with no contract checks, all checks at runtime, and gradual checking.
- The evaluation **should** compare an LLM agent's hidden-test pass rate across the three conditions, with the same model, prompt and budget and several runs per task, following a pilot study.

2.2 Non-Functional Requirements

- **Soundness:** a contract classified as proven **must** never fail at runtime, tested with property-based testing.
- **Correctness:** the compiler **must** be covered by an automated test suite.
- **Reproducibility:** contracts and task selection **must** be fixed before any experimental runs, with model settings and all runs logged.
- **Cost:** experiments **must** stay within an agreed budget.

2.3 Out of Scope

Classes, asynchronous code, modules, closures, exceptions, mutable objects other than arrays, and aliasing, the full standard library, higher-order contracts, and production-scale systems.

The above requirements are subject to change as the project evolves.

3 Project Plan

3.1 Gantt Chart

3.2 Tasks and Milestones

3.3 Risk Assessment

4 Resources

References

Bader, J., Aldrich, J., and Tanter, É., 2018. *Gradual program verification* [Online]. Available from: https://doi.org/10.1007/978-3-319-73721-8_2 [Accessed November 1, 2026].

Dijkstra, E.W., 1970. *Notes on structured programming* [Online]. (technical report). Technological University Eindhoven. Available from: https://www.cs.utexas.edu/~EWD/ewd02xx/EWD249.PDF [Accessed November 1, 2026].

Dijkstra, E.W., 1972. The humble programmer. *Communications of the acm* [Online]. Available from: https://www.cs.utexas.edu/~EWD/transcriptions/EWD03xx/EWD340.html [Accessed November 1, 2026].

Findler, R.B. and Felleisen, M., 2002. *Contracts for higher-order functions* [Online]. Available from: https://doi.org/10.1145/581478.581484 [Accessed November 1, 2026].

GitHub, 2026. *Octoverse: commits, pushes, reach new highs in 2026 as developers build with agents* [Online]. Available from: https://github.blog/news-insights/octoverse/octoverse-commits-pushes-reach-new-highs-in-2026-as-developers-build-with-agents/ [Accessed November 1, 2026].

Gleirscher, M. and Marmsoler, D., 2020. *Formal methods in dependable systems engineering: a survey of professionals from Europe and North America* [Online]. Available from: https://doi.org/10.1007/s10664-020-09836-5 [Accessed November 1, 2026].

Hoare, C.A.R., 1969. *An axiomatic basis for computer programming* [Online]. Available from: https://doi.org/10.1145/363235.363259 [Accessed November 1, 2026].

Klein, G., Elphinstone, K., Heiser, G., Andronick, J., Cock, D., Derrin, P., Elkaduwe, D., Engelhardt, K., Kolanski, R., Norrish, M., Sewell, T., Tuch, H., and Winwood, S., 2009. *seL4: formal verification of an OS kernel* [Online]. Available from: https://trustworthy.systems/publications/papers/Klein_EHACDEEKNSTW_09.pdf [Accessed November 1, 2026].

Kleppmann, M., 2025. *Prediction: AI will make formal verification go mainstream* [Online]. Available from: https://martin.kleppmann.com/2025/12/08/ai-formal-verification.html [Accessed November 1, 2026].

Leino, K.R.M., 2010. *Dafny: an automatic program verifier for functional correctness* [Online]. Available from: https://doi.org/10.1007/978-3-642-17511-4_20 [Accessed November 1, 2026].

Meyer, B., 1992. *Applying "design by contract"* [Online]. Available from: https://doi.org/10.1109/2.161279 [Accessed November 1, 2026].

Møller, A. and Schwartzbach, M.I., 2026. *Static program analysis* [Online]. Available from: https://cs.au.dk/~amoeller/spa/ [Accessed November 2, 2026].

Moura, L. de and Bjørner, N., 2008. *Z3: an efficient SMT solver* [Online]. Available from: https://doi.org/10.1007/978-3-540-78800-3_24 [Accessed November 2, 2026].

Stack Overflow, 2026. *2026 developer survey: AI* [Online]. Available from: https://survey.stackoverflow.co/2026/ai/data [Accessed October 31, 2026].

Vero, M., Mündler, N., Chibotaru, V., Raychev, V., Baader, M., Jovanović, N., He, J., and Vechev, M., 2025. *BaxBench: can LLMs generate correct and secure backends?* [Online]. Available from: https://baxbench.com/ [Accessed November 1, 2026].

Wang, S., 2026. *VibeContract: the missing quality assurance piece in vibe coding* [Online]. Available from: https://arxiv.org/abs/2603.15691 [Accessed November 2, 2026].

Wang, Y., Pradel, M., and Liu, Z., 2025. *Are "solved issues" in SWE-bench really solved correctly? An empirical study* [Online]. Available from: https://arxiv.org/abs/2503.15223 [Accessed November 1, 2026].